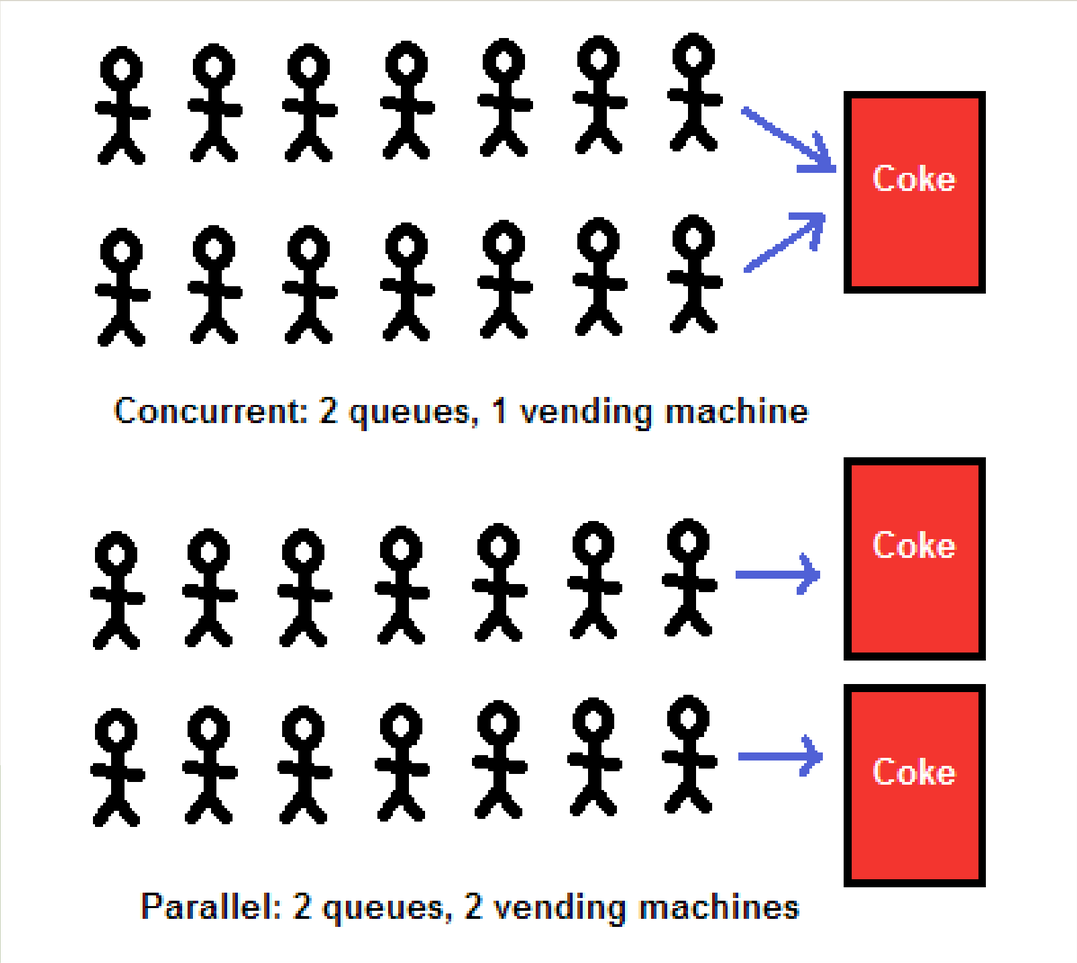
Coding Club, IIT Guwahati

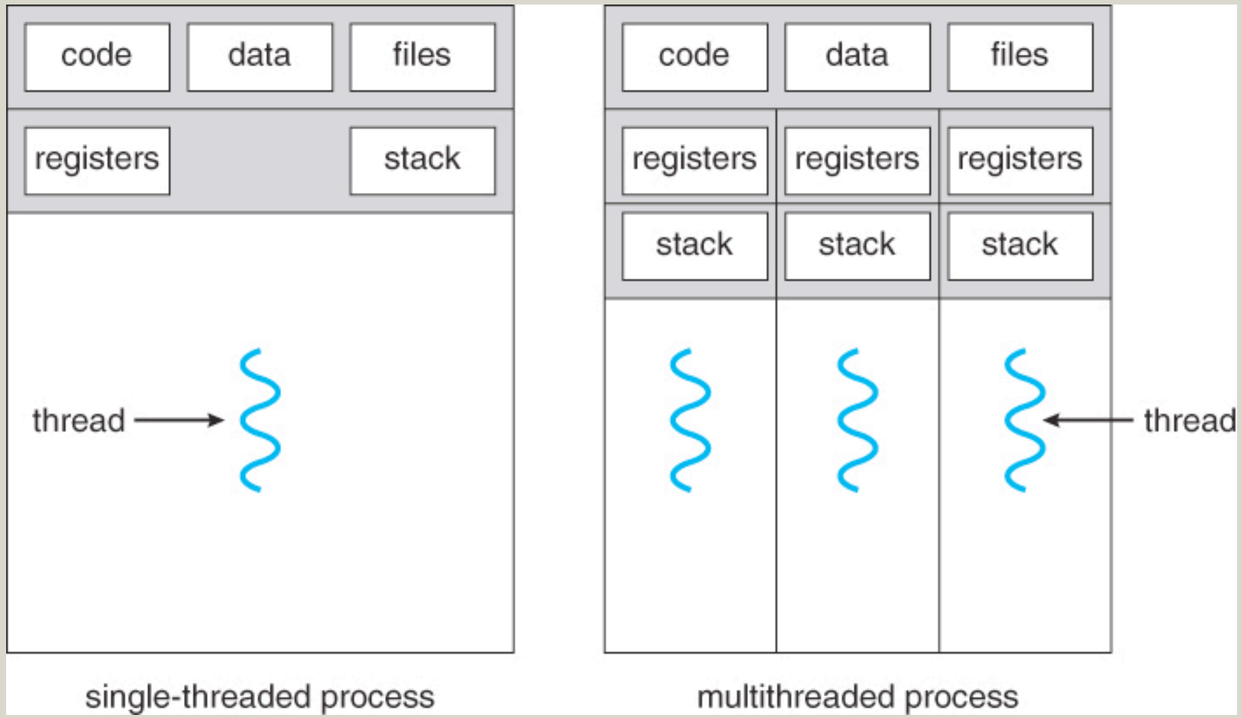
C++ Multithreading features

Session 2 : Shared data, race conditions, std::mutex and more



Toshit Jain

What are threads ?



A thread is the smallest unit of execution within a process, representing a single sequence of instructions that can run independently. Multiple threads within the same process share the same memory space, resources, and code, but each thread has its own program counter, registers, and stack. This allows threads to run concurrently, improving performance and responsiveness, especially on multi-core systems, while requiring careful synchronization to avoid **data races** and inconsistencies.

Threads share memory

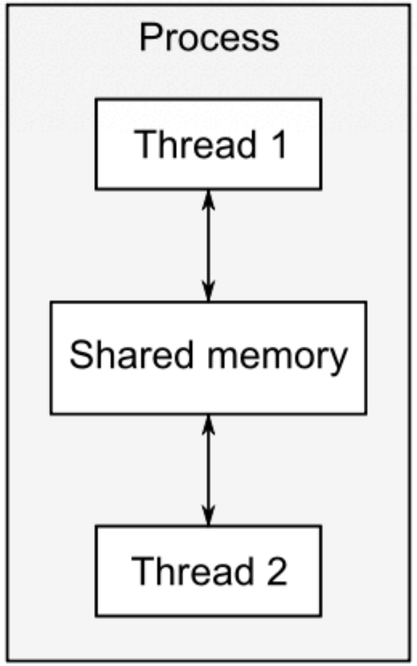


Figure 1.4 Communication between a pair of threads running concurrently in a single process

Problems with shared data

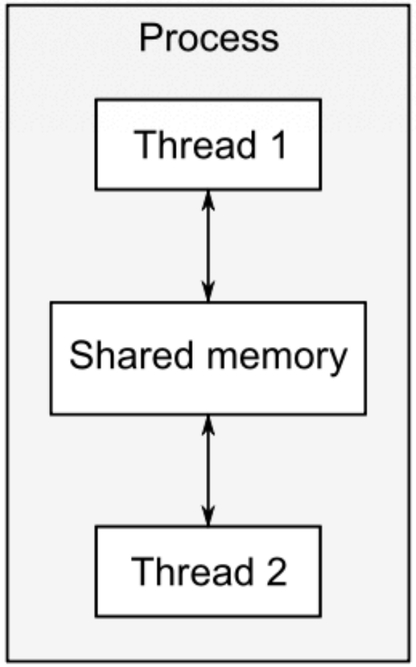


Figure 1.4 Communication between a pair of threads running concurrently in a single process

- If all shared data is read-only, there's no problem, because the data read by one thread is unaffected by whether or not another thread is reading the same data. However, if data is shared between threads, and one or more threads start modifying the data, there's a lot of potential for trouble.

- When we have one or more thread(s) modifying the shared data, there is a possibility of **race condition**. In concurrency, a race condition is anything where the outcome depends on the relative ordering of execution of operations on two or more threads; the threads race to perform their respective operations.

- The C++ Standard also defines the term data race to mean the specific type of race condition that arises because of concurrent modification to a single object, **data races** cause the **undefined behavior**.

Example of data race

Thread A

```
if(!q.empty()){

  int value = q.front();

  q.pop();
  do_something(value)
}
```

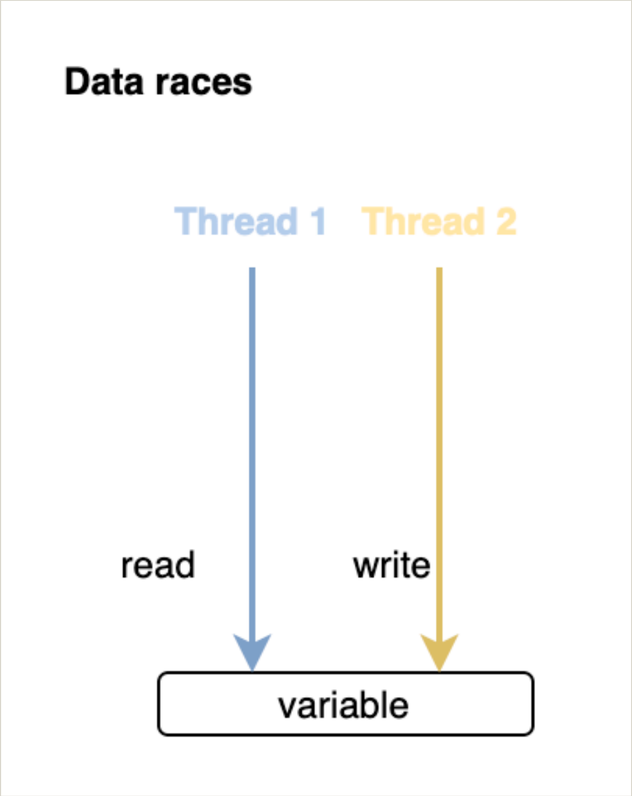
Thread B

```

if(!q.empty()){

  int value = q.front();

  q.pop();
  do_something(value)
}
```

Data race between two threads using shared std::queue object

For more examples, read Ch. 3 from C++ Concurrency in Action

Resolution of race condition



1. The simplest option is to wrap your data structure with a protection mechanism, to ensure that only the thread actually performing a modification can see the intermediate states where the invariants are broken. (**lock-based approach**) → uses **std::mutex**

2. Another option is to modify the design of your data structure and its invariants so that modifications are done as a series of indivisible changes, each of which preserves the invariants. This is generally referred to as **lock-free programming** and is difficult to get right. → uses **std::atomic**

3. Another way of dealing with race conditions is to handle the updates to the data structure as a **transaction**, just as updates to a database are done within a transaction. The required series of data modifications and reads is stored in a transaction log and then **committed** in a single step. (Will study in OS in Sem 5 not used in C++ as of yet)

std::mutex & Mutual Exclusion

main

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <queue>

std::mutex m;
std::queue<int> q;

void push_to_queue(int val){
  std::lock_guard<std::mutex> lk(m);
  q.push(val);
}

void get_from_queue(int& a){
  std::lock_guard<std::mutex> lk(m);
  a = q.top();
  q.pop();
}
```

1. One of the best way to resolve a data race problem is to make a data structur **mutually exclusive** using **mutex**. To access the data structure, you first **lock** the mutex. While it is locked, no other thread can use the data structure (thus making it mutually exclusive). Once it is **unlocked** by this thread, it can then be used by another thread.

2. To use **std::mutex** you use the **mutex::lock()** and **mutex::unlock()** functions. A better way to do this is to use **std::lock_guard** instead, which takes a **std::mutex** object and locks it across the scope, similar to the **thread_guard** we developed earlier.

3. In general it is good to club data member and mutex into a class(private), rather than keeping them global as a good design practice.

A potential implementation flaw

main

```
#include <iostream>
#include <thread>
#include <mutex>
#include <queue>

class Queue{
private:
  std::queue<int> q;
  std::mutex m;
public:
  template <typename callable>
  wrong_function(callable func){
    std::lock_guard<std::mutex> lk(m);
    func(q);
  }
} obj;

std::queue<int>* malicious_func(void);

int main(){
  std::queue<int>* q = obj.wrong_function(malicious_func)
}
```

1. One of the major implementation flaws which happens a lot of times is a function which **returns pointer or reference** to the protected data. In that way, we have made a hole in our protection, cause now the user can use the data without ever locking the mutex.

2. As much as it is important to not return pointers/ references, it is also important to not pass any **callable which acts on the shared data as an argument**. This callable can then, suppose have pointers/references and can return the same to the data.

3. *Don't pass pointers and references to protected data outside the scope of the lock, whether by returning them from a function, storing them in externally visible memory, or passing them as arguments to user-supplied functions.*

Making our very own threadsafe queue :)

main ×

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <queue>

std::mutex m;
std::queue<int> q;

void push_to_queue(int val){
  std::lock_guard<std::mutex> lk(m);
  q.push(val);
}

void get_from_queue(int& a){
  std::lock_guard<std::mutex> lk(m);
  a = q.top();
  q.pop();
}
```

1. One of the best way to resolve a data race problem is to make a data structur **mutually exclusive** using **mutex**. To access the data structure, you first **lock** the mutex. While it is locked, no other thread can use the data structure (thus making it mutually exclusive). Once it is **unlocked** by this thread, it can then be used by another thread.

2. To use **std::mutex** you use the **mutex::lock()** and **mutex::unlock()** functions. A better way to do this is to use **std::lock_guard** instead, which takes a **std::mutex** object and locks it across the scope, similar to the **thread_guard** we developed earlier.

3. In general it is good to club data member and mutex into a class(private), rather than keeping them global as a good design practice.

Making our very own threadsafe queue :)

main ×

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <queue>

std::mutex m;
std::queue<int> q;

void push_to_queue(int val){
  std::lock_guard<std::mutex> lk(m);
  q.push(val);
}

void get_from_queue(int& a){
  std::lock_guard<std::mutex> lk(m);
  a = q.top();
  q.pop();
}
```

Onto VS Code

Making our very own threadsafe queue :)

main

```cpp
#include <iostream>
#include <mutex>
#include <queue>

template<typename T>
class TSFQ{
private:
    std::queue<T> q;
    std::mutex m;
public:
    TSFQ() {}
    // Remove copy constructor and = operator
    // ...
    void push(T value){
        std::lock_guard<std::mutex> lk(m);
        q.push(value);
    }
    bool try_pop(T& value){
        std::lock_guard<std::mutex> lk(m);
        if(q.empty()) return false;
        value = q.front();
        q.pop();
        return true;
    }
    std::shared_ptr<T> try_pop(){
        std::lock_guard<std::mutex> lk(m);
        if(q.empty()) return nullptr;
        std::shared_ptr<T> res(std::make_shared<T>(q.front()));
        q.pop();
        return res;
    }
    bool empty(){
        std::lock_guard<std::mutex> lk(m);
        return q.empty();
    }
};
```

1. One **std::mutex** object to lock the queue while performing any operation on the queue.

2. Overriden **pop()** for small and large data types. Small can return using parameterized while large can do using **std::shared_ptr**

3. Templated for generalized use.

4. **Copy constructor** and **Assignment operator** removed because std::mutex is not copy constructible. Can add modified copy constructor (will work).

5. Coarse-grained locking (lock whole queue at a time)

Coarse v/s fine grained locking

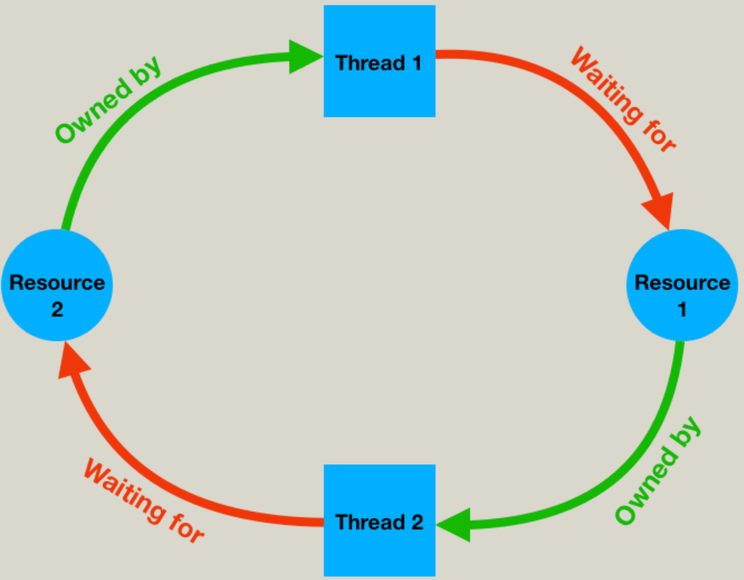
Coarse Grained Locking means that we use less mutexes in general and lock those mutexes during each operation.

The first versions of the Linux kernel that were designed to handle multi-processor systems used a single global kernel lock. Although this worked, it meant that a two-processor system typically had much worse performance than two single-processor systems.

Fine grained locking is when you use multiple mutexes, where each is used for some set of operations, and lock them only when those operations are invoked.

Why not then use fine grained locking everywhere? The issue is Deadlock!! It can happen that thread 1 locks mutex 1, and thread 2 locks mutex 2 and now both the threads are waiting for the other mutex to be released by the contrary to start their processing.

Deadlock and How to prevent it ?



Following are the ways to prevent deadlock in C++ :

1. By using **std::lock** - atomically locks both the mutexes in a thread.

2. **Avoid nested locks** - The first idea is the simplest: don't acquire a lock if you already hold one. If you need to acquire multiple locks, do it as a single action with std::lock.

3. **Avoid calling user supplied code while holding a mutex** - Because the code is user supplied, you have no idea what it could do; it could do anything, including acquiring a lock.

4. **Follow a specific order while locking** - If you absolutely must acquire two or more locks, and you can't acquire them as a single operation with std::lock, the next-best thing is to acquire them in the same order in every thread.

5. **Use a lock heirarchy** - Give each mutex a heirarchy status, and store the current heirarchy of code before locking. (Read more from Ch.3)

Using std::lock

main ×

```
class some_big_object;
void swap(some_big_object& lhs,some_big_object& rhs);
class X{
private:
  some_big_object some_detail;
  std::mutex m;
public:
  X(some_big_object const& sd):some_detail(sd){}
  friend void swap(X& lhs, X& rhs){
    if(&lhs==&rhs)
    return;
    std::lock(lhs.m,rhs.m);
    std::lock_guard<std::mutex> lock_a(lhs.m,std::adopt_lock);
    std::lock_guard<std::mutex> lock_b(rhs.m,std::adopt_lock);
    swap(lhs.some_detail,rhs.some_detail);
  }
};
```

The **std::adopt_lock** parameter is supplied in addition to the mutex to indicate to the **std::lock_guard** objects that the mutexes are already locked, and they should just adopt the ownership of the existing lock on the mutex rather than attempt to lock the mutex in the constructor.

Other methods to avoid deadlock

main ×

```cpp
class some_class;

some_class data1;
std::mutex m1;
some_class data2;
std::mutex m2;

void func1(){
  // do something
  std::lock_guard<std::mutex> lk1(m1);
  std::lock_guard<std::mutex> lk2(m2);
  // do something
}

void func2(){
  // do something
  std::lock_guard<std::mutex> lk1(m1);
  std::lock_guard<std::mutex> lk2(m2);
  // do something
}

int main(){
  std::thread t1(func1);
  std::thread t2(func2);
  t1.join(); t2.join();
}
```

Keeping same order while locking (point 4)

main ×

```cpp
// heirarchical_mutex class uses low unsigned for low level stuff and high unsigned for higher
// Once a low level stuff is locked, a higher level cannot be locked
// Thus prevents deadlock by forcing order of locks
class heirarchical_mutex{
private:
    mutex m;
    unsigned heirarchy_value;
    unsigned previous_heirarchy_value;
    // static thread_local makes it a static variable per thread
    static thread_local unsigned thread_heirarchy_value;
public:
    // No implicit conversion allowed
    explicit heirarchical_mutex(unsigned value) : heirarchy_value(value), previous_heirarchy_value(0) {}
    void lock(){
        // Acquire lock only with value less than current thread
        if(thread_heirarchy_value <= heirarchy_value) {
            std::__throw_logic_error("Mutex heirarchy violated");
        }
        m.lock();
        previous_heirarchy_value = thread_heirarchy_value;
        thread_heirarchy_value = heirarchy_value;
    }
    void unlock(){
        thread_heirarchy_value = previous_heirarchy_value;
        m.unlock();
    }
    bool try_lock(){
        if(thread_heirarchy_value <= heirarchy_value) {
            std::__throw_logic_error("Mutex heirarchy violated");
        }
        if(!m.try_lock()) return false;
        previous_heirarchy_value = thread_heirarchy_value;
        thread_heirarchy_value = heirarchy_value;
    }
};
thread_local unsigned heirarchical_mutex::thread_heirarchy_value = UINT_MAX;
```

Heirarchial mutex (point 5)

std::unique_lock - a better std::lock_guard

1. The **std::lock_guard** locks the mutex during construction and unlocks during the destruction when it goes out of scope. However, **std::unique_lock** on the other hand can modify this.

2. It does not take the ownership of **std::mutex** and can **lock()/unlock()** on will.

3. We can also use **std::defer_lock()** which tells the **std::unique_lock** that we will manually lock the mutex later, so do not lock on construction please.

4. It also has a **owns_lock()** function which returns the flag stating whether the object owns the lock at this moment or not. During destruction, the **owns_lock()** flag is checked, and if it is true, then the lock is unlocked.

5. We can also return a **std::unique_lock** object from a function. This then means the ownership of lock is transferred to the callee and thus the destructor is also called once the caller ends. The object is **movable** and **not copyable**.

6. **Everything merry? Absolutely not**. Since we have so many more flexibility, the **std::unique_lock** uses much more space than a **std::lock_guard** object. So if you can use a **std::lock_guard** object anywhere, use it!

std::unique_lock implementation

Onto VS Code

We will see a killer use of **std::unique_lock** with **std::condition_variable** in the next session.

Locking suggestions

1. The lock granularity is a hand-waving term to describe the amount of data protected by a single lock. A fine-grained lock protects a small amount of data, and a coarse-grained lock protects a large amount of data.

2. In particular, don't do any really time-consuming activities like file I/O while holding a lock. File I/O is typically hundreds (if not thousands) of times slower than reading or writing the same volume of data from memory. So unless the lock is really intended to protect access to the file, performing I/O while holding the lock will delay other threads unnecessarily (because they'll block while waiting to acquire the lock), potentially eliminating any performance gain from the use of multiple threads. In general, a lock should be held for only the minimum possible time needed to perform the required operations.

3. We can also do more fine grained locking, however it can so happen that due to more fine grained locking, you are comparing object1 at time t1, to object2 at time t2, which can lead to race conditions. eg. front() in a multithreaded queue, you get the front object however, it might change before you process it (front() might have been popped many times by other threads). This is thus called an unsafe peek.

Additional Points

- There are more locking methods instead of std::mutex which you can see in **section 3.3 of C++ Concurrency in Action.** However these are not that standard and important. Obviously there are some special use cases for each but those are too specific to be studied as of now.

All the things were taken from...

- C++ Concurrency in Action by Anthony Williams :

1. Chapter 3 : Sharing data between threads

Next session...

- Synchronization and Condition variables

Thank You